

FRED TALKS

3rd October 2026

CONTENTS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

the senior engineer death spiral

SUNIL PAI · 959 WORDS

Mastering Codex Remote for engineering

DEVELOPERS.OPENAI.COM · 2207 WORDS

Ownership

THORSTEN BALL · 1122 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · [SOURCE](#)

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments...). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs... but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

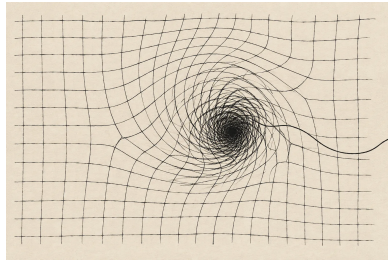
In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

the senior engineer death spiral

SUNIL PAI · 19 SEP 2026 · [SOURCE](#)

(a friend recently got a new job, a very senior role, very well paid, and pretty different from his previous gig. he was asking me how to do a 60 to 80-hour week, how to do well enough to get promoted, etc. and as we talked, I realised he was feeling a bit of imposter syndrome and really wanted to prove himself. I ended up giving him a bit of a monologue. this is more or less what I told him.)



A hand-drawn grid spiraling into a dense knot, with one loose line escaping to the edge.

okay, so this is the most common failure mode I've seen. I call it the senior engineer death spiral. it usually happens when an engineer goes into a new job, gets a promotion, or even just gets a big project at work. or they ask for a big project because they think to themselves, "oh, if I work harder, do a bigger thing, then I will be rewarded with promotions and what have you," right? like, that's the move. and the thing they do is they tell themselves they need to almost cosplay being a more senior engineer than they are. they're like, "oh, you know what, I'm going to try designing something way more ambitious."

and what happens is they start disappearing for longer periods of time. like, for two, three weeks, you won't hear anything from them. occasionally, during the standup, they'll give what I call the positive update. "yeah, things are going well, you guys. I'll have something to show you quite soon. if you have any questions, reach out." and no one usually reaches out, but they won't really have anything to show.

and in the back of their head, right, they've started the spiral already. they're telling themselves, "oh my god, I haven't really shipped anything in a while. what I need to do is work harder. I'll just work hard enough. I'll save this and no one will know." like, "in the next week, I'll do a whole month's work. and no one will know." they'll start losing sleep, all of that. and it sucks, because bro, they start getting depressed, missing meals. their relationships suffer, both work and personal. and they just start spiraling, spiraling, and it ends up in worst-case scenarios. they burn out, they need to take a month off, they get PIP'd, they get fired, they quit their jobs because they think, "you know what, this is not salvageable." and it really sucks.

and by the way, this isn't something that I've noticed only with other people. this has happened to me. and it's happened so many times that I now recognize it early enough to take a step back and fix it.

see, man, the bigger point here is that because of COVID and coding agents and remote work, things that have happened in the last five to ten years, right, you don't really have the same structure of working next to someone and being given Jira tasks, etc. you have a lot more

ownership, and you are unfortunately a little more siloed. so you never want to be in a situation where people don't know what's up with you. like, they shouldn't be saying, "yeah, I don't know what Sunil's been doing," and starting to get concerned, because they might only tell you when it's too late. you want to be in a position where "Sunil doesn't shut the fuck up about what he's working on," right? again, you don't have to be obnoxious about it. you have to share your work.

I think in moments like this, the thing you need to do is actually a little counterintuitive. but you have to start from a position of assuming that everyone around you is operating in good faith, okay? like, they hired you for you. they didn't hire you for who you think you're going to be in six months. they like you for who you are right now, bro.

so the thing to do is actually to drop a level. not to try to be a higher-level engineer, but to almost drop a level and simply become the greatest teammate for a while. you're still responsible for your work, you're just trying to get moving again. you know, "I'm just here to help and support my teammates. I'm going to do bugs, the annoying things on somebody else's plate or that no one has been able to get to. do grunt work, organizational stuff, write-ups. just things to help people out."

and the reason you want to do this is you want to move away from an outcome-based mindset to momentum-based. like, you want to build a routine, because momentum is everything in this, and relationships are an even bigger thing in this. and by getting momentum going and fixing your relationships with other people, the thing you get is stamina. and that's when you realize the real lesson, which is that big projects are not done with big efforts. it's a big marathon, just slow, steady, incremental work. forget about a weekly basis, like a daily basis, an hourly basis, to the point that it's muscle memory.

you're in automatic mode. you wake up, you make yourself coffee, you get to work, and you just do the work. and then 30 days later, you look back and you see the immense volume of work that you've done just by doing a little bit every day. because the thing you're going for is reliability, fixing your relationships. you want to take the burden off your teammates and your manager, because then, once you build that trust, they will give you bigger work. that's the move here.

remember, you are in the reputation-building business, and software is actually downstream of that. godspeed and best of luck.

Mastering Codex Remote for engineering

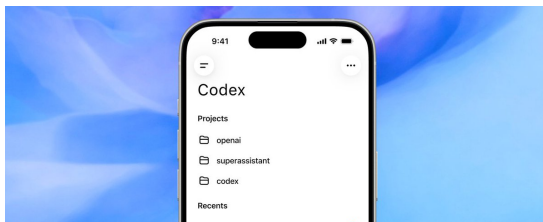
DEVELOPERS.OPENAI.COM · 25 JUN 2026 · [SOURCE](#)

Codex Remote is easy to underestimate.

At first glance, it looks like a way to check on a coding task from your phone. That is useful, but it misses the bigger idea. The real power of Codex Remote is that it lets you start, direct, review, and organize work running on your development machines without pretending that an iPhone should be a tiny terminal.

Over the last two months, we have added a surprising amount of depth to that loop: remote host connections, worktrees, goals, side chats, inline code review, queued and steering prompts, attachments, skills and plugins, archived threads, security controls, and a long tail of small details that make the app useful for serious work.

This is the field guide I wish every new power user had.



The right mental model: Your phone is the control plane

The code still runs where it belongs: on your Mac, Windows machine, devbox, or other connected host. Codex Remote gives you a native interface for controlling that work.

That distinction matters. The goal is not to reproduce every terminal affordance on a small screen. The goal is to make the decisions that unblock an agent easy from anywhere:

- What repository and workspace should it use?
- Should this run in the current branch or a fresh worktree?

- Should my next message wait, or should it redirect the active turn?
- Is this command safe to approve?
- What changed, and do I agree with it?
- Should this become a durable goal, a separate thread, or a quick side question?

Once you use the app this way, it stops feeling like remote desktop software and starts feeling like an engineering control plane.

1. Start the task with the right boundaries

Good agent work starts with a correctly scoped environment. Codex Remote lets you choose the connected host and workspace before the first prompt. For a new thread, you can also choose a branch, create a separate worktree, and run the environment setup associated with it.

That makes a few useful patterns possible:

- Use the current checkout for a quick investigation.
- Create a new worktree for a change that should stay isolated.
- Start from the intended base branch instead of fixing Git state later.
- Let the environment setup run before asking Codex to build or test.

This is one of the most important power-user habits: Spend 10 seconds choosing the right execution context and save 10 minutes of cleanup later.

The composer can also carry more context than plain text. You can attach files, photos, or a fresh camera capture. Skills and plugins appear inline, which makes it much easier to confirm that your prompt is invoking the capability you intended.

My rule is simple: If a screenshot, file, or named skill can remove ambiguity, attach or mention it before the first turn.

2. Learn the difference between Queue and Steer

This is probably the least obvious high-leverage setting in Codex Remote.

When Codex is already working, a follow-up can do one of two things:

- **Queue** waits until the current response finishes, then sends your prompt as the next turn.
- **Steer** injects guidance into the work already in progress.

Queue is the safe default. Use it for a second task, an extra test request, or anything that should happen after the current work.

Steer is for correcting direction while the cost of continuing down the wrong path is still growing:

- | Keep the fix inside the mobile package. Do not refactor the shared renderer.
- | The reproduction only happens after reconnect. Test the resumed path, not the live path.
- | Stop investigating the UI. Check whether the server removed the item during resume.

That turns the phone into something more useful than a status screen. You can intervene at the exact moment a run needs judgment.

You can choose the default follow-up behavior in Codex settings. I keep Queue as the default and use Steer deliberately; accidental mid-turn redirection is usually more expensive than waiting.

3. Use side chats as a branch of thought

Long-running coding threads accumulate valuable context. Interrupting one with every side question makes the main transcript noisy and can pull the agent away from the objective.

That is what side chats solve.

Use `/side` to open a lightweight conversation connected to the current thread. Use `/side <prompt>` to open it with a question already prepared. Even better, select text in the transcript and choose **Ask in side chat**. The selected passage becomes the starting context for the new conversation.

I use side chats for questions such as:

- Why did Codex choose this architecture?

- What does this error actually mean?
- Is this behavior consistent with the desktop app?
- Give me a release-note version of this implementation detail.
- What should I verify before I approve this command?

The distinction is useful: The main thread owns the work; the side chat helps me understand the work.

4. Use Plan for the path and Goal for the outcome

Plan mode and goals solve different problems.

Plan mode asks Codex to propose the implementation path before changing code. It is useful when the task is underspecified, risky, or likely to touch several systems.

A goal is durable. It tells Codex what outcome to keep pursuing across turns. On mobile, `/goal` can create and manage that objective, while progress remains visible as the work continues.

A practical pattern is:

1. Start with Plan mode for a risky change.
2. Inspect the proposed boundaries.
3. Turn the accepted outcome into a goal when the work will require multiple iterations.
4. Let Codex continue through implementation, tests, review feedback, and cleanup without restating the objective every time.

Plans answer “How should we approach this?” Goals answer “What must be true before we are done?”

5. Review code without leaving the conversation

The review loop is where Codex Remote becomes genuinely useful for engineering rather than merely convenient.

Completed turns can show a summary of changed files. From there, you can open the diff, inspect individual files, expand or collapse sections, wrap long lines, and open a source file with syntax highlighting. You can attach inline comments to the relevant lines and send that review context back to Codex.

There are several levels to the workflow:

- Open the changed-file summary for a quick sanity check.
- Tap through to the full source when a diff lacks surrounding context.
- Add inline comments for precise corrections.
- Use the review command to review local changes or compare against a branch.
- Link a file back into the chat when you want Codex to reason about that file specifically.

This makes a tight mobile review loop possible:

1. Codex completes the implementation.
2. I inspect the diff from my phone.
3. I leave two inline comments.
4. Codex addresses them in the same thread.
5. I review the smaller follow-up diff.

The important point is not that a phone replaces a large monitor for deep code reading. It does not. The point is that many reviews are blocked on one or two decisions, and those decisions no longer need to wait until I am back at my desk.

6. Treat permissions as part of the workflow

Remote work is only useful if control stays explicit.

Codex Remote surfaces approval requests for commands, file changes, network access, and connected tools. Depending on the request and host configuration, an approval may apply once, for the current chat, or more broadly.

The power-user move is not to approve everything. It is to choose the narrowest permission that keeps the work moving.

For a well-understood command in a trusted thread, a chat-scoped approval can remove repetitive interruptions. For an unfamiliar command, a sensitive repository, or a request whose effect is unclear, approve once—or deny it and ask Codex to explain or use a safer approach.

You can also choose the thread's broader approval behavior when starting work. Think of this as part of task setup, alongside the host, workspace, branch, and model.

7. Manage context before it becomes a problem

Agent threads are stateful, and long threads eventually accumulate enough context to become slower or less focused.

Codex Remote has a small set of tools for managing that lifecycle:

- `/status` shows session details, workspace, context usage, and available rate-limit information.
- The optional context indicator keeps remaining context visible in the composer.
- `/compact` compresses an overgrown thread while preserving the useful working state.
- `/fork` creates a new thread from the current one when you want shared history but a different direction.

The practical sequence is: Check status, compact when the objective is still the same, and fork when the objective has diverged.

Do not use a side chat and a fork interchangeably. A side chat is a lightweight question around the current work. A fork is a new primary line of work that inherits the thread's history.

8. Keep a clean Thread Desk

My own Codex workflow increasingly looks like a small operations desk.

I pin the few threads that are actively moving, rename them around outcomes, and archive them aggressively once the work is done. Archived-thread browsing makes this safe: Archive is organization, not deletion.

Notifications are part of that system. A completion notification can open the relevant Codex task directly, so the handoff from “agent finished” to “human reviews” is one tap.

Spotlight and Shortcuts can open Codex Remote directly. On iPad, keyboard shortcuts make the app surprisingly efficient: Create a new thread, move between threads, open changed files, pin, rename, and archive without reaching for every control.

This is the “chief of staff” use case for Codex Remote. The app is not only where I issue prompts. It is where I keep track of which pieces of engineering work are active, blocked, waiting for review, or finished.

9. The hidden command palette

Typing `/` reveals the fastest route to many of the app’s advanced behaviors. Availability can depend on the connected host, app version, and account configuration.

Command	What it is for
<code>/plan</code>	Toggle Plan mode before implementation.
<code>/goal <objective></code>	Create or update a durable objective.
<code>/side [question]</code>	Ask a side question without disrupting the main thread.
<code>/review</code>	Review local changes or compare them with a branch.
<code>/status</code>	Inspect the session, workspace, context, and rate limits.
<code>/compact</code>	Compress a long thread’s context.
<code>/fork</code>	Create a new primary thread from the current history.
<code>/fast</code>	Switch between standard and faster execution when available.
<code>/feedback</code>	Send product feedback tied to the current session.

The command palette is worth learning because it exposes the product’s real conceptual model: Plan, pursue, branch, review, inspect, and recover.

10. Five workflows that work especially well on mobile

The release captain

Start a focused thread for a release or pull request. Ask Codex to check the current branch, CI state, open review feedback, and release inclusion. Pin the thread. When new information arrives, steer only when it invalidates the active investigation; otherwise, queue it. Review the final diff or release note from the phone and archive the thread when the release lands.

The interrupt-driven bug fix

Attach a screenshot, log, or captured file. Ask Codex to diagnose before editing. Use a side chat to interrogate one suspicious error without derailing the main investigation. Once the cause is clear, return to the main thread and authorize the narrow fix.

The mobile reviewer

Run a review against the intended branch, inspect the changed-file summary, open the files that matter, and attach inline comments. Ask Codex to address only those comments, then review the follow-up diff.

The long-running objective

Create a goal with a concrete completion condition: Tests green, review feedback resolved, or a reproducible performance threshold met. Check progress from notifications and status rather than repeatedly asking “Are you done?” Use queued prompts for additional work and reserve steering for corrections.

The multi-machine operator

Keep hosts named clearly and organize work by machine and workspace. Use the phone to start a task on the machine that has the right checkout, credentials, simulator, or operating system. This is especially useful when one task requires a Mac and another needs a Windows host.

Small features with outsized impact

Some of my favorite additions are not headline features:

- Edit the latest sent prompt instead of adding a correction turn.

- Save or copy rendered images from a task.
- See queued prompts while Codex is still running.
- Open completed tasks directly from notifications.
- Browse archived threads instead of keeping the active list crowded.
- Wrap long diff lines when reading on a narrow screen.
- Use Face ID or the device passcode to protect Codex separately.
- See skills and plugins inline in the composer before sending.
- Capture a photo directly into a prompt when the real-world object is the context.
- Start a side chat from selected transcript text rather than re-explaining the question.

None of these changes the fundamental agent model. Together, they remove the friction that makes a remote workflow feel fragile.

The larger lesson

The best mobile software does not shrink a desktop interface. It identifies the decisions that matter when you are away from the desk and makes those decisions fast, legible, and safe.

That is how I think about Codex Remote now. It is where I can choose the right environment, set the objective, redirect a run, answer an approval, inspect the result, and keep the whole queue of engineering work coherent.

The computer still does the work. The phone keeps me in control.

Ownership

THORSTEN BALL · 08 JUL 2026 · [SOURCE](#)

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.



Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase “ownership” or allude to it, but haven’t explained what “ownership” means to me in a while.

So, ownership.

If I ask you “can you own this?” or “can you take care of this?” or “are you on it?” — what I’m doing is I’m asking you to *own* it, to own the solution of a problem from end to end. From “we have a problem” to “we don’t have to think about it again.”

That means, when you say that you’re owning something, the expectation is that you...

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we’re actually trying to solve here. Maybe you think “the problem is that we need to migrate from using X to using Y”, but that’s not a problem, that’s a solution. The problem is likely something like “performance is bad”, “it’s not stable”, “it fails for customer x”. Maybe there’s other possible solutions to that? Think about those. What are the tradeoffs? What’s the best solution to go with considering these tradeoffs?
- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?
- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?

- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven't confirmed yet?
- Think about how you'd test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?
- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!
- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say “here's what I built, under these constraints, with these tradeoffs?”
- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?
- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?
- If you think your colleagues need to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.
- Do customers need to know? Who reported the bug? Who's blocked? Let them know.
- Does the world need to know? Announce that it's out.
- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A

department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.



“How does this apply juniors? You can’t expect them to really do all of that?”

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, ... — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.



You know what *you* should own? A subscription to this newsletter: